

1 Assignment Grading Policy

- **Late Submissions:** Any assignment submitted after the deadline on the LMS will receive a maximum of 70% of the total score.
- **Code Functionality:** Points will be deducted if the code does not run properly, produces invalid results, or does not follow the required interface.
- **Reproducibility:** Your program must be executable from the command line. If the TA cannot reproduce your results, points may be deducted.

2 Overview

2.1 Background Reading

For this assignment, you are encouraged to read the following paper:

- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

You are not required to fully reproduce the paper, but you should understand the following key idea:

- very deep neural networks are difficult to optimize,
- residual connections help gradient flow and make training easier,
- ResNet can achieve better performance than plain deep networks.

2.2 Objective

The goal of this assignment is to train a **ResNet-based image classifier** on the **CIFAR-100** dataset and experience a more advanced deep learning training pipeline.

Through this assignment, you will:

- load and preprocess CIFAR-100,
- train a ResNet model for image classification,
- compare **pretrained** initialization and **training from scratch**,
- use a learning-rate scheduler,
- apply data augmentation,
- visualize training behavior using learning curves,
- analyze why residual connections are useful in deep neural networks.

2.3 Dataset Description

CIFAR-100 consists of 60,000 color images of size 32×32 in 100 classes.

- 50,000 training images
- 10,000 test images

Compared with CIFAR-10, CIFAR-100 is more difficult because:

- it contains many more classes,
- each class has fewer training examples,
- many categories are visually similar.

2.4 Why This Assignment?

In Assignment 1, you trained an FCNN on CIFAR-10 and learned how to manage experiments. In Assignment 2, you trained a simple CNN and compared it with FCNN. In this assignment, you will move to a deeper and more powerful model, **ResNet**, and study how modern image classification pipelines are organized.

3 Assignment Tasks

3.1 Task 1: Data Loading and Preprocessing

Load CIFAR-100 using a deep learning framework such as PyTorch.

Required preprocessing:

- `ToTensor()`
- `Normalize(...)`

Required augmentation:

- Use at least **two** augmentation methods for the training set.

Examples:

- random crop,
- random horizontal flip,
- color jitter,
- random erasing.

You must also split the original training set into:

- training set,
- validation set.

3.2 Task 2: Use a ResNet Model

Use a ResNet-based model such as ResNet18 or ResNet34.

Requirements:

- You may use `torchvision.models`.
- You must modify the final classification layer so that the output dimension is 100.
- You must clearly explain your architecture choice in the report.

3.3 Task 3: Compare Pretrained and Scratch Initialization

You must compare the following two settings:

- **Pretrained**: initialize the model using pretrained weights,
- **Scratch**: initialize the model randomly and train from scratch.

Your report must compare:

- convergence speed,
- validation accuracy,
- final test accuracy,
- advantages and disadvantages of each setting.

3.4 Task 4: Use a Learning-Rate Scheduler

Your training code must support a scheduler.
You may choose one of the following:

- StepLR,
- MultiStepLR,
- CosineAnnealingLR.

Your report must explain:

- which scheduler you used,
- why you selected it,
- how it affected training behavior.

3.5 Task 5: Use Argument Parser

Your program must support command-line arguments using argparse.
At minimum, support the following arguments:

- -model
- -batch_size
- -lr
- -epochs
- -optimizer
- -scheduler
- -pretrained
- -save_csv

You may additionally include:

- -weight_decay
- -num_workers
- -seed
- -save_dir

Example:

```
python train.py --model resnet18 --batch_size 128 --lr 0.001 \
--epochs 30 --optimizer adam --scheduler cosine --pretrained 1
```

3.6 Task 6: Run Multiple Experiments

You must run at least **4 experiments** in total.
Your experiments must include:

- at least one **pretrained** run,
- at least one **scratch** run,
- at least one scheduler comparison,
- at least one optimizer or augmentation comparison.

You are encouraged to automate your experiments using a shell script such as `run_resnet_experiments.sh`.

3.7 Task 7: Save Results to CSV

Your program must save experiment results to a CSV file.
At minimum, each row should contain:

- model name,
- pretrained or scratch,
- batch size,
- learning rate,
- number of epochs,
- optimizer,
- scheduler,
- validation accuracy,
- test accuracy.

3.8 Task 8: Plot Learning Curves

For the **best-performing configuration based on validation accuracy**, you must plot the following curves over training epochs:

- training loss,
- validation loss,
- validation accuracy.

These curves must be included in your report.

Important: Do **not** evaluate the test set at every epoch. The test set must be used **only once** for the final evaluation of the best model selected using the validation set.

3.9 Task 9: Residual Connection Analysis

In your report, answer the following questions:

1. What is a residual connection?
2. Why are residual connections useful in deep networks?
3. How is ResNet conceptually different from the simple CNN used in Assignment 2?
4. What do your learning curves suggest about optimization difficulty and generalization?

3.10 Task 10: Performance Analysis

Your report must include at least one of the following:

- top-1 accuracy,
- top-5 accuracy,
- class-wise accuracy,
- confusion matrix for selected classes.

4 Implementation Requirements

4.1 Required Files

Your submission must include:

- `train.py` or `main.py`
- `run_resnet_experiments.sh`
- `results.csv`
- `report.pdf`

4.2 Training Output

Your code should print or save:

- training loss,
- validation loss,
- validation accuracy,
- final test accuracy.

5 Deliverables

5.1 Source Code

Submit all source files required to run and reproduce your experiments.

5.2 Report

Submit a short report (**3–4 pages recommended**) including:

- model description,
- explanation of preprocessing and augmentation,
- explanation of pretrained vs. scratch settings,
- explanation of scheduler usage,
- table of experiments and results,
- learning curves,
- residual connection analysis,
- discussion of the best-performing configuration.

5.3 Result Table

Your report must include a table similar to the following:

Model	Init.	Optimizer	Scheduler	LR	Val Acc.	Test Acc.
ResNet18	Scratch	SGD	StepLR	0.1	–	–
ResNet18	Pretrained	SGD	StepLR	0.01	–	–
ResNet18	Scratch	Adam	Cosine	0.001	–	–
ResNet18	Pretrained	Adam	Cosine	0.001	–	–

6 Grading Rubric

- **Correct use of CIFAR-100 and ResNet pipeline (25 pts)**
- **Pretrained vs. scratch comparison (20 pts)**
- **Scheduler and experiment design (15 pts)**
- **Learning curves and result analysis (15 pts)**
- **Residual connection explanation (15 pts)**
- **Code quality and report quality (10 pts)**

7 Important Notes

- You may use pretrained backbones from standard libraries.
- Do not simply report accuracy values. You must explain and interpret your results.
- Use the validation set for model selection and hyperparameter tuning.
- Use the test set only for final evaluation.
- A well-organized experimental pipeline is as important as final model performance.